

University of Science and Technology in Zewail City

CIE Program

CIE 555: Neural Networks and Deep Learning

Course Project

Grammar Error Correction (GEC)

Instructor: Dr. Ibrahim swelam

Teaching Assistant:

Eng. Mahmoud Farahat

Eng. Aya Abdelaziz

Prepared by:

Abdelhady Mohamed 202201172

Abdulrahman Magdy 202200341

Contents

1	Introduction	3
1.1	Project Overview	3
1.2	Objectives	3
1.3	Scope of the Work	4
2	Dataset and Data Preparation	4
2.1	Overview of the C4 Dataset	4
2.2	Selected C4 Subset	4
2.3	Data Sampling Strategy	5
2.4	Dataset Cleaning and Filtering	5
2.5	Shuffling and Balancing	5
2.6	Train, Validation, and Test Split	5
2.7	Text Preprocessing	5
2.8	Tokenization and Vocabulary Construction	5
2.9	Vectorization Pipeline	6
3	Evaluation Methodology	6
3.1	Evaluation Metrics	6
3.2	Token-Level Accuracy	6
3.3	Precision, Recall, and F0.5 Score	6
3.4	Why F0.5 is Important for GEC	7
3.5	Inference Strategy	7
4	Model 1: Bag-of-Words Feedforward Network	7
4.1	Model Architecture	7
4.2	Training Configuration	7
4.3	Advantages and Limitations	8
4.4	Experimental Results	8
5	Model 2: LSTM Sequence-to-Sequence without Attention	8
5.1	Encoder-Decoder Architecture	8
5.2	Embedding Layer	9
5.3	Bidirectional LSTM Encoder	9
5.4	Decoder and Teacher Forcing	9
5.5	Training Configuration	9
5.6	Advantages and Limitations	9
5.7	Experimental Results	10
6	Model 3: LSTM Sequence-to-Sequence with Bahdanau Attention	10
6.1	Attention Mechanism	10
6.2	Encoder Architecture	10
6.3	Decoder Architecture	10
6.4	Bahdanau Attention Layer	11
6.5	Training Configuration	11
6.6	Advantages and Limitations	11
6.7	Experimental Results	12

7	Model 4: Transformer Encoder-Decoder	12
7.1	Transformer Architecture	12
7.2	Positional Encoding	12
7.3	Multi-Head Self-Attention	12
7.4	Encoder and Decoder Blocks	13
7.5	Learning Rate Warmup Strategy	13
7.6	Training Configuration	13
7.7	Advantages and Limitations	13
7.8	Experimental Results	14
8	Comparative Analysis of Models	14
8.1	Comparison of Architectures	14
8.2	Training Time and Computational Cost	14
8.3	Performance Comparison	14
8.4	Metric-Based Evaluation	15
8.5	Strengths and Weaknesses of Each Model	15
8.6	Best Performing Model	15
9	Grammar-Rule-Based Analysis	15
9.1	Verb Tense Corrections	15
9.2	Subject-Verb Agreement	15
9.3	Article and Determiner Corrections	16
9.4	Preposition Corrections	16
9.5	Spelling and Word Choice Errors	16
9.6	Sentence Structure Improvements	16
9.7	Failure Cases and Error Patterns	16
10	Final Challenge and Dataset Quality Discussion	17
10.1	Challenges Faced During Training	17
10.2	Memory and Computational Constraints	17
10.3	Sequence Length Challenges	17
10.4	Dataset Noise and Imperfections	17
10.5	Impact of Dataset Quality on Performance	17
10.6	Generalization Challenges	18
10.7	Possible Improvements	18
11	Conclusion	18
11.1	Summary of Findings	18
11.2	Key Achievements	18
11.3	Lessons Learned	19
11.4	Future Work	19

1 Introduction

1.1 Project Overview

Grammar Error Correction (GEC) is an important task in Natural Language Processing (NLP) that focuses on automatically detecting and correcting grammatical mistakes in text. GEC systems are widely used in applications such as writing assistants, educational platforms, machine translation post-processing, and intelligent proofreading tools.

The goal of this project is to design, implement, train, and evaluate multiple deep learning architectures for the Grammar Error Correction task using a large-scale dataset derived from the C4 corpus. The project investigates how different neural architectures perform when learning grammatical transformations from noisy input sentences to corrected target sentences.

Four different models were implemented and compared throughout the project:

- Bag-of-Words Feedforward Neural Network (BoW-FNN)
- LSTM Sequence-to-Sequence Model
- LSTM Sequence-to-Sequence with Bahdanau Attention
- Transformer Encoder-Decoder Model

Each model was trained under similar preprocessing conditions and evaluated using multiple metrics including token-level accuracy, precision, recall, and the F0.5 score, which is commonly used in Grammar Error Correction research.

The project also includes qualitative grammar-rule-based analysis to examine how well each architecture handles different categories of grammatical errors such as verb tense correction, subject-verb agreement, spelling mistakes, and sentence restructuring.

1.2 Objectives

The main objectives of this project are summarized as follows:

- Build a large-scale Grammar Error Correction dataset from the C4 corpus.
- Design and implement multiple deep learning architectures for sequence correction tasks.
- Compare traditional sequence models with attention-based and Transformer-based architectures.
- Evaluate model performance using quantitative and qualitative analysis techniques.
- Investigate the effect of attention mechanisms on grammatical correction quality.
- Analyze the strengths and weaknesses of each model architecture.
- Study the impact of dataset quality and preprocessing on final model performance.
- Explore computational challenges associated with training large-scale NLP models.

1.3 Scope of the Work

This project focuses specifically on English Grammar Error Correction using supervised deep learning techniques. The work includes the complete machine learning pipeline starting from dataset preparation and preprocessing up to model training, inference, evaluation, and comparative analysis.

The scope of the project includes:

- Data extraction and preprocessing from the C4 dataset.
- Construction of training, validation, and testing datasets.
- Implementation of four different neural network architectures.
- Training and optimization of all models using TensorFlow and Keras.
- Development of custom inference pipelines for autoregressive decoding.
- Performance evaluation using standard GEC metrics.
- Linguistic and grammar-rule-based analysis of prediction outputs.
- Discussion of dataset limitations, training challenges, and computational constraints.

The project does not focus on:

- multilingual grammar correction,
- speech-based grammar correction,
- reinforcement learning approaches,
- or deployment optimization for production systems.

Instead, the primary emphasis is placed on understanding and comparing different sequence modeling architectures for Grammar Error Correction under large-scale training conditions.

2 Dataset and Data Preparation

2.1 Overview of the C4 Dataset

This project used a subset of the C4 (Colossal Clean Crawled Corpus) dataset, a large English text corpus collected from Common Crawl. The dataset contains diverse sentence structures suitable for Grammar Error Correction (GEC) tasks.

2.2 Selected C4 Subset

Due to hardware limitations, a subset of approximately 1 million sentence pairs was used. Each pair contained:

- Incorrect input sentence
- Corrected target sentence

2.3 Data Sampling Strategy

The dataset was created by sampling:

100,000 samples from each of 10 files

The selected data was then shuffled randomly before training.

2.4 Dataset Cleaning and Filtering

The preprocessing steps included:

- Removing missing values
- Cleaning whitespace
- Converting text to string format
- Removing identical input-target pairs

The final dataset contained approximately 999,994 samples.

2.5 Shuffling and Balancing

Random shuffling improved batch diversity and reduced ordering bias. Equal sampling from multiple files improved dataset balance and generalization.

2.6 Train, Validation, and Test Split

The dataset was divided into:

- Training: 747,994 samples
- Validation: 102,000 samples
- Testing: 150,000 samples

2.7 Text Preprocessing

The preprocessing pipeline included:

- Lowercasing
- Punctuation normalization
- Adding `starttoken` and `endtoken`

2.8 Tokenization and Vocabulary Construction

TensorFlow's `TextVectorization` layer was used with a vocabulary size of:

10,000 tokens

2.9 Vectorization Pipeline

Text sequences were converted into fixed-length integer vectors with maximum sequence lengths of 100 tokens.

Chunk-based vectorization was used to avoid out-of-memory errors during preprocessing.

3 Evaluation Methodology

3.1 Evaluation Metrics

The models were evaluated using multiple token-level metrics to measure correction quality and prediction accuracy. The selected metrics were:

- Token-Level Accuracy
- Precision
- Recall
- F0.5 Score

These metrics provide both quantitative performance evaluation and linguistic correction quality assessment.

3.2 Token-Level Accuracy

Token-level accuracy measures how many predicted tokens exactly match the target tokens.

$$Accuracy = \frac{CorrectTokens}{TotalTokens}$$

This metric gives a general indication of prediction correctness but may not fully reflect grammatical correction quality.

3.3 Precision, Recall, and F0.5 Score

Precision measures how many predicted corrections were correct:

$$Precision = \frac{CorrectPredictions}{TotalPredictedTokens}$$

Recall measures how many target corrections were successfully recovered:

$$Recall = \frac{CorrectPredictions}{TotalTrueTokens}$$

The F0.5 score combines precision and recall while giving higher importance to precision:

:contentReference[oaicite:0]index=0

3.4 Why F0.5 is Important for GEC

In Grammar Error Correction, incorrect corrections are often more harmful than missing corrections. Therefore, precision is more important than recall.

The F0.5 score is commonly used in GEC research because it weights precision higher than recall, making it more suitable for evaluating grammatical correction systems.

3.5 Inference Strategy

All sequence-based models used greedy decoding during inference. The decoder generates one token at a time until:

- the `endtoken` is produced,
- the maximum sequence length is reached,
- or repeated token patterns are detected.

Special tokens such as padding, unknown, and start tokens were blocked during prediction to improve output quality.

4 Model 1: Bag-of-Words Feedforward Network

4.1 Model Architecture

The first model was a simple Bag-of-Words Feedforward Neural Network (BoW-FNN) used as a baseline model.

The architecture consisted of:

- Sparse Bag-of-Words input vector
- Dense hidden layer with ReLU activation
- Dropout layer
- RepeatVector layer
- TimeDistributed output layer with Softmax activation

The model ignores word order and only learns token frequency patterns.

4.2 Training Configuration

The model was trained using:

- Adam optimizer
- Sparse categorical cross-entropy loss
- Batch size of 512
- Early stopping and learning rate reduction

To avoid memory overflow, sparse matrix generators were used during training.

4.3 Advantages and Limitations

Advantages:

- Fast training
- Low computational cost
- Simple implementation

Limitations:

- Ignores word order
- Cannot model sentence structure
- Weak contextual understanding

4.4 Experimental Results

The model achieved limited performance compared to sequence-based architectures.

The final evaluation results were approximately:

Metric	Score
Accuracy	0.035
Precision	0.0495
Recall	0.0682
F0.5 Score	0.0523

The results confirmed that Bag-of-Words models are not suitable for complex Grammar Error Correction tasks.

5 Model 2: LSTM Sequence-to-Sequence without Attention

5.1 Encoder-Decoder Architecture

The second model used a Seq2Seq architecture based on LSTM networks. It consisted of:

- Bidirectional LSTM encoder
- LSTM decoder
- Dense output layer with Softmax activation

The encoder converts the input sentence into hidden states, while the decoder generates the corrected sentence token by token.

5.2 Embedding Layer

Separate embedding layers were used for both encoder and decoder inputs.

The embedding dimension was set to:

128

Embeddings allowed the model to learn semantic relationships between words.

5.3 Bidirectional LSTM Encoder

A Bidirectional LSTM encoder was used to capture both past and future context in the sentence.

The forward and backward hidden states were concatenated and projected into decoder states using dense layers.

5.4 Decoder and Teacher Forcing

The decoder generated output sequences one token at a time.

Teacher forcing was applied during training, where the correct previous token was provided to the decoder instead of the predicted token. This improved convergence speed and training stability.

5.5 Training Configuration

The model was trained using:

- Adam optimizer
- Sparse categorical cross-entropy loss
- Batch size of 64
- Early stopping
- Model checkpoint saving

The model was trained for 5 epochs.

5.6 Advantages and Limitations

Advantages:

- Captures sequential information
- Learns sentence structure
- Better contextual understanding than BoW

Limitations:

- Encoder bottleneck problem
- Difficulty handling long sequences
- Slower training compared to Transformer models

5.7 Experimental Results

The LSTM Seq2Seq model achieved significantly better performance than the baseline model.

Metric	Score
Accuracy	0.1098
Precision	0.3364
Recall	0.3302
F0.5 Score	0.3351

The results showed that sequence modeling greatly improves Grammar Error Correction performance.

6 Model 3: LSTM Sequence-to-Sequence with Bahdanau Attention

6.1 Attention Mechanism

The third model extended the Seq2Seq LSTM architecture by adding Bahdanau Attention.

Instead of compressing the entire input sentence into a single vector, the decoder dynamically attends to all encoder hidden states during generation.

This improves handling of long and complex sentences.

6.2 Encoder Architecture

The encoder used:

- Embedding layer
- Bidirectional LSTM
- Dense projection layers

The encoder returned both sequence outputs and hidden states for the attention mechanism.

6.3 Decoder Architecture

The decoder consisted of:

- Decoder embedding layer
- LSTM decoder
- Context vector concatenation
- Dense Softmax output layer

At each decoding step, the decoder received both the current token embedding and the attention context vector.

6.4 Bahdanau Attention Layer

The Bahdanau Attention layer calculates alignment scores between:

- decoder hidden states,
- and encoder outputs.

The attention weights are normalized using Softmax to produce a context vector representing important input positions.

This allows the model to focus on relevant words during correction generation.

6.5 Training Configuration

The model was trained using:

- Adam optimizer
- Sparse categorical cross-entropy loss
- Batch size of 64
- Early stopping
- Learning rate reduction
- Model checkpoint saving

The attention model required longer training time due to sequential decoding operations.

6.6 Advantages and Limitations

Advantages:

- Better handling of long sentences
- Improved contextual learning
- Higher correction quality

Limitations:

- Higher computational cost
- Slower inference speed
- More memory usage

6.7 Experimental Results

The attention-based model achieved better results than the standard Seq2Seq LSTM.

Metric	Score
Accuracy	0.1632
Precision	0.4543
Recall	0.4354
F0.5 Score	0.4504

The results demonstrated the importance of attention mechanisms in Grammar Error Correction tasks.

7 Model 4: Transformer Encoder-Decoder

7.1 Transformer Architecture

The final model was a Transformer-based encoder-decoder architecture.

Unlike LSTM models, the Transformer processes all tokens in parallel using self-attention mechanisms, allowing faster training and better handling of long-range dependencies.

The architecture included:

- Token and positional embeddings
- Transformer encoder blocks
- Transformer decoder blocks
- Final Dense Softmax output layer

7.2 Positional Encoding

Since Transformers do not process sequences recurrently, positional encoding was added to token embeddings to preserve word order information.

Sinusoidal positional encoding was used to represent token positions within sequences.

7.3 Multi-Head Self-Attention

The Transformer used Multi-Head Self-Attention to learn relationships between all words in the sentence simultaneously.

This mechanism allows the model to capture:

- grammatical dependencies,
- contextual meaning,
- and long-range sentence structure.

7.4 Encoder and Decoder Blocks

The encoder blocks contained:

- Multi-Head Attention
- Feedforward layers
- Residual connections
- Layer normalization

The decoder blocks additionally used masked attention to prevent access to future tokens during generation.

7.5 Learning Rate Warmup Strategy

A custom warmup learning rate schedule was used during training.

The learning rate gradually increased during early training steps before decreasing smoothly. This improved training stability and convergence.

7.6 Training Configuration

The model was trained using:

- Adam optimizer with warmup scheduling
- Sparse categorical cross-entropy loss
- Batch size of 64
- Early stopping
- Model checkpoint saving

The Transformer model trained significantly faster than the attention-based LSTM despite its strong performance.

7.7 Advantages and Limitations

Advantages:

- Parallel processing
- Strong contextual understanding
- Excellent long-sequence handling
- Fast training speed

Limitations:

- Higher GPU memory usage
- More complex architecture
- Requires large datasets for best performance

7.8 Experimental Results

The Transformer achieved the best overall performance among all models.

Metric	Score
Accuracy	0.2116
Precision	0.5612
Recall	0.5245
F0.5 Score	0.5535

The Transformer also achieved a validation accuracy of approximately 92.56% during training, showing strong learning capability and stable convergence.

The results confirmed that Transformer architectures are highly effective for large-scale Grammar Error Correction tasks.

8 Comparative Analysis of Models

8.1 Comparison of Architectures

The four models used different approaches for Grammar Error Correction.

- The BoW-FNN model relied on token frequency only.
- The LSTM Seq2Seq model learned sequential dependencies.
- The Attention model improved context handling using Bahdanau Attention.
- The Transformer used self-attention and parallel processing.

As model complexity increased, correction quality also improved.

8.2 Training Time and Computational Cost

The BoW model trained the fastest with the lowest memory usage.

The LSTM models required longer training due to sequential processing, while the Attention model was the most computationally expensive.

The Transformer achieved the best balance between speed and performance because of parallel computation.

8.3 Performance Comparison

Model	Accuracy	Precision	Recall	F0.5
BoW-FNN	0.0350	0.0495	0.0682	0.0523
LSTM Seq2Seq	0.1098	0.3364	0.3302	0.3351
LSTM + Attention	0.1632	0.4543	0.4354	0.4504
Transformer	0.2116	0.5612	0.5245	0.5535

8.4 Metric-Based Evaluation

The Transformer achieved the highest scores across all evaluation metrics.

The Attention-based LSTM significantly improved performance compared to the standard Seq2Seq model, showing the importance of attention mechanisms in GEC tasks.

The BoW model performed poorly because it could not capture sentence structure or contextual relationships.

8.5 Strengths and Weaknesses of Each Model

BoW-FNN:

- Fast but weak contextual understanding.

LSTM Seq2Seq:

- Better sequence learning but suffers from encoder bottleneck limitations.

LSTM + Attention:

- Improved contextual focus and better long-sequence handling.

Transformer:

- Best contextual learning and strongest correction performance.

8.6 Best Performing Model

The Transformer Encoder-Decoder model achieved the best overall results.

Its self-attention mechanism allowed the model to capture long-range grammatical dependencies more effectively than recurrent architectures.

The final results confirmed that Transformer-based architectures are the most suitable solution for large-scale Grammar Error Correction tasks in this project.

9 Grammar-Rule-Based Analysis

9.1 Verb Tense Corrections

The models successfully corrected several verb tense errors such as:

- “She go to school” → “She goes to school”
- “He have a car” → “He has a car”

The Transformer and Attention models performed best in tense correction tasks.

9.2 Subject-Verb Agreement

The sequence-based models learned subject-verb agreement patterns effectively.

Common corrections included:

- singular/plural agreement,
- auxiliary verb correction,
- and verb form normalization.

9.3 Article and Determiner Corrections

The models corrected missing or incorrect articles such as:

- “a”, “an”, and “the”

The Transformer model produced the most grammatically natural determiner usage.

9.4 Preposition Corrections

Several preposition errors were corrected successfully, including:

- incorrect word choice,
- missing prepositions,
- and unnecessary insertions.

Attention-based models showed stronger contextual understanding in these cases.

9.5 Spelling and Word Choice Errors

The models corrected many spelling mistakes and noisy tokens.

Examples included:

- misspelled words,
- repeated tokens,
- and incorrect word substitutions.

The Transformer demonstrated the best semantic correction quality.

9.6 Sentence Structure Improvements

The advanced models improved sentence fluency by:

- reordering words,
- removing redundant phrases,
- and generating more natural sentence structures.

These improvements were limited in the BoW baseline model.

9.7 Failure Cases and Error Patterns

Despite good overall performance, several challenges remained:

- over-correction,
- repeated words,
- incomplete sentences,
- and incorrect semantic substitutions.

The LSTM models occasionally struggled with long sentences, while the Transformer sometimes generated grammatically correct but semantically inaccurate outputs.

10 Final Challenge and Dataset Quality Discussion

10.1 Challenges Faced During Training

Training large-scale GEC models required significant computational resources and careful memory management. Several training runs needed optimization to avoid instability and runtime interruptions.

10.2 Memory and Computational Constraints

The 1M-sample dataset created high RAM and GPU memory usage during preprocessing and training.

To reduce memory issues:

- chunk-based vectorization was used,
- sparse generators were applied for the BoW model,
- and batch sizes were adjusted dynamically.

10.3 Sequence Length Challenges

Long sentences increased computational cost and made learning more difficult for recurrent models.

The standard LSTM model particularly suffered from information bottlenecks on long sequences, while the Transformer handled long-range dependencies more effectively.

10.4 Dataset Noise and Imperfections

The C4-derived dataset contained noisy web text, including:

- spelling mistakes,
- incomplete sentences,
- duplicated patterns,
- and inconsistent grammar quality.

Some sentence pairs were also semantically weak despite grammatical similarity.

10.5 Impact of Dataset Quality on Performance

Dataset quality strongly affected model performance.

Noisy or poorly aligned sentence pairs sometimes caused:

- incorrect corrections,
- semantic drift,
- and unstable predictions.

Cleaner datasets would likely improve correction accuracy further.

10.6 Generalization Challenges

Some models struggled when handling:

- rare grammatical structures,
- unseen vocabulary,
- and highly complex sentences.

The Transformer showed the strongest generalization capability among all models.

10.7 Possible Improvements

Future improvements may include:

- larger vocabulary sizes,
- deeper Transformer architectures,
- beam search decoding,
- better dataset filtering,
- and pretrained language model initialization.

11 Conclusion

11.1 Summary of Findings

This project explored four different deep learning architectures for Grammar Error Correction using a large-scale dataset derived from the C4 corpus.

The results showed that model performance improved significantly as architectures became more advanced, with the Transformer model achieving the best overall performance.

11.2 Key Achievements

The main achievements of the project include:

- Building a 1M-sample GEC dataset,
- Implementing four different NLP architectures,
- Training large-scale models successfully,
- and performing both quantitative and linguistic evaluation.

11.3 Lessons Learned

The project demonstrated that:

- attention mechanisms are highly important for GEC tasks,
- dataset quality strongly affects model performance,
- and Transformer architectures outperform traditional recurrent models in large-scale sequence correction tasks.

11.4 Future Work

Future work may focus on:

- pretrained Transformer models,
- larger and cleaner datasets,
- advanced decoding strategies,
- and multilingual Grammar Error Correction systems.